

Лекция. Введение в SQL

Узнайте, как использовать SQL для чтения и манипулирования данными в реляционной базе данных.

Использование SQL для чтения и записи в базу данных

Изучите основные концепции реляционных баз данных и научитесь использовать SQL для чтения и работы с базой данных.

Использование SQL для чтения и записи в базу данных

Прежде чем начать

Многие из используемых вами приложений хранят данные непосредственно на устройстве.

Приложение «Часы» хранит повторяющиеся будильники, приложение «Карты Google» сохраняет список последних поисков, а приложение «Контакты» позволяет добавлять, редактировать и удалять информацию о контактах.

Постоянство данных - хранение или сохранение данных на устройстве - является важной частью разработки Android. Благодаря постоянству данных пользовательский контент не теряется при закрытии приложения, а данные, загруженные из Интернета, сохраняются, чтобы их не нужно было загружать заново.

SQLite - это распространенный способ хранения данных, предоставляемый Android SDK для Android-приложений. SQLite представляет собой реляционную базу данных, которая позволяет представлять данные аналогично тому, как вы структурируете данные с помощью классов Kotlin. В этом коделабе изучаются основы языка SQL-Structured Query Language, который, хотя и не является настоящим языком программирования, обеспечивает простой и гибкий способ чтения и модификации базы данных SQLite с помощью всего нескольких строк кода.

Получив фундаментальные знания SQL, вы будете готовы использовать библиотеку Room для добавления постоянства в ваши приложения, о чем мы расскажем далее в этом разделе.

Примечание: В приложениях для Android есть несколько способов хранения данных, включая внутренние и внешние хранилища. В этом разделе рассматриваются Room и Preferences Datastore. Чтобы узнать больше о различных методах хранения данных в Android, обратитесь к обзору хранения данных и файлов.

Ключевые понятия реляционных баз данных

Что такое база данных? Если вы знакомы с программой для работы с электронными таблицами, например Google Sheets, то вы уже знакомы с базовой аналогией базы данных.

Электронная таблица состоит из отдельных таблиц данных или отдельных электронных таблиц в одной рабочей книге.



Каждая таблица состоит из столбцов, которые определяют, что представляют собой данные, и строк, в которых представлены отдельные элементы со значениями для каждого столбца. Например, вы можете определить столбцы для идентификатора, имени, специальности и оценки студента.

	A	B	C	D
1	id	name	major	gpa

Каждая строка содержит данные об одном студенте со значениями для каждого из столбцов.

	A	B	C	D
1	id	name	major	gpa
2	1	Maha	Mechanical Engineering	4
3	2	Geraldine	Physics	3.92
4	3	Ishaan	Linguistics	3.64

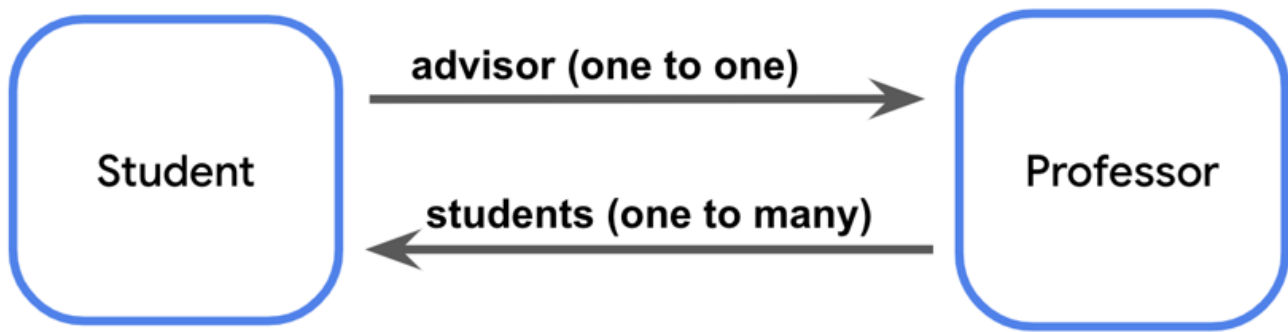
Реляционная база данных работает аналогичным образом.

Таблицы определяют высокоуровневые группы данных, которые вы хотите представить, например, студенты и преподаватели. Столбцы определяют данные, которые содержит каждая строка таблицы. Строки содержат фактические данные, состоящие из значений для каждого столбца таблицы. Структура реляционной базы данных также отражает то, что вы уже знаете о классах и объектах в Kotlin.

```
data class Student(
    id: Int,
    name: String,
    major: String,
    gpa: Double
)
```

Классы, как и таблицы, моделируют данные, которые вы хотите представить в своем приложении. Свойства, например столбцы, определяют конкретные части данных, которые должен содержать каждый экземпляр класса. Объекты, как и строки, представляют собой фактические данные. Объекты содержат значения для каждого свойства, определенного в классе, так же как строки содержат значения для каждого столбца, определенного в таблице данных. Как электронная таблица может содержать несколько листов, а приложение - несколько классов, так и база данных может содержать несколько таблиц. База данных называется реляционной, если она может моделировать отношения между таблицами. Например, аспирант может иметь одного профессора в качестве консультанта по

докторской диссертации, в то время как этот профессор является консультантом по докторской диссертации для нескольких студентов.



Каждая таблица в реляционной базе данных содержит уникальный идентификатор для строк, например, столбец, значение которого в каждой строке является автоматически увеличивающимся целым числом. Этот идентификатор известен как первичный ключ.

Когда таблица ссылается на первичный ключ другой таблицы, это называется внешним ключом. Наличие внешнего ключа означает, что между таблицами существует связь.

Примечание: Как и в случае с классами Kotlin, принято использовать форму единственного числа для названия таблиц базы данных. Для приведенного выше примера это означает, что вы называете таблицы `teacher`, `student` и `course`, а не множественные формы `teachers`, `students` и `courses`.

Что такое SQLite? SQLite - это широко используемая реляционная база данных. Точнее, SQLite - это облегченная библиотека на языке Си для управления реляционными базами данных с помощью языка структурированных запросов, известного как SQL и иногда произносимого как «сиквел» для краткости.

Для работы с реляционной базой данных вам не придется изучать C или какой-либо совершенно новый язык программирования. SQL - это просто способ добавлять и извлекать данные из реляционной базы данных с помощью нескольких строк кода.

Примечание: Не все базы данных организованы в виде таблиц, столбцов и строк. Другие виды баз данных, известные как NoSQL, структурированы аналогично объекту JSON с вложенными парами ключей и значений. Примерами баз данных NoSQL являются Redis или Cloud Firestore.

Представление данных с помощью SQLite В Kotlin вы знакомы с такими типами данных, как `Int` и `Boolean`. Базы данных SQLite тоже используют типы данных! Колонки таблицы данных должны иметь определенный тип данных. В следующей таблице приведено соответствие распространенных типов данных Kotlin их эквивалентам в SQLite.

Kotlin data type	SQLite data type
Int	INTEGER
String	VARCHAR or TEXT
Boolean	BOOLEAN
Float, Double	REAL

Таблицы в базе данных и столбцы в каждой таблице называются схемой. В следующем разделе вы загрузите стартовый набор данных и узнаете больше о его схеме.

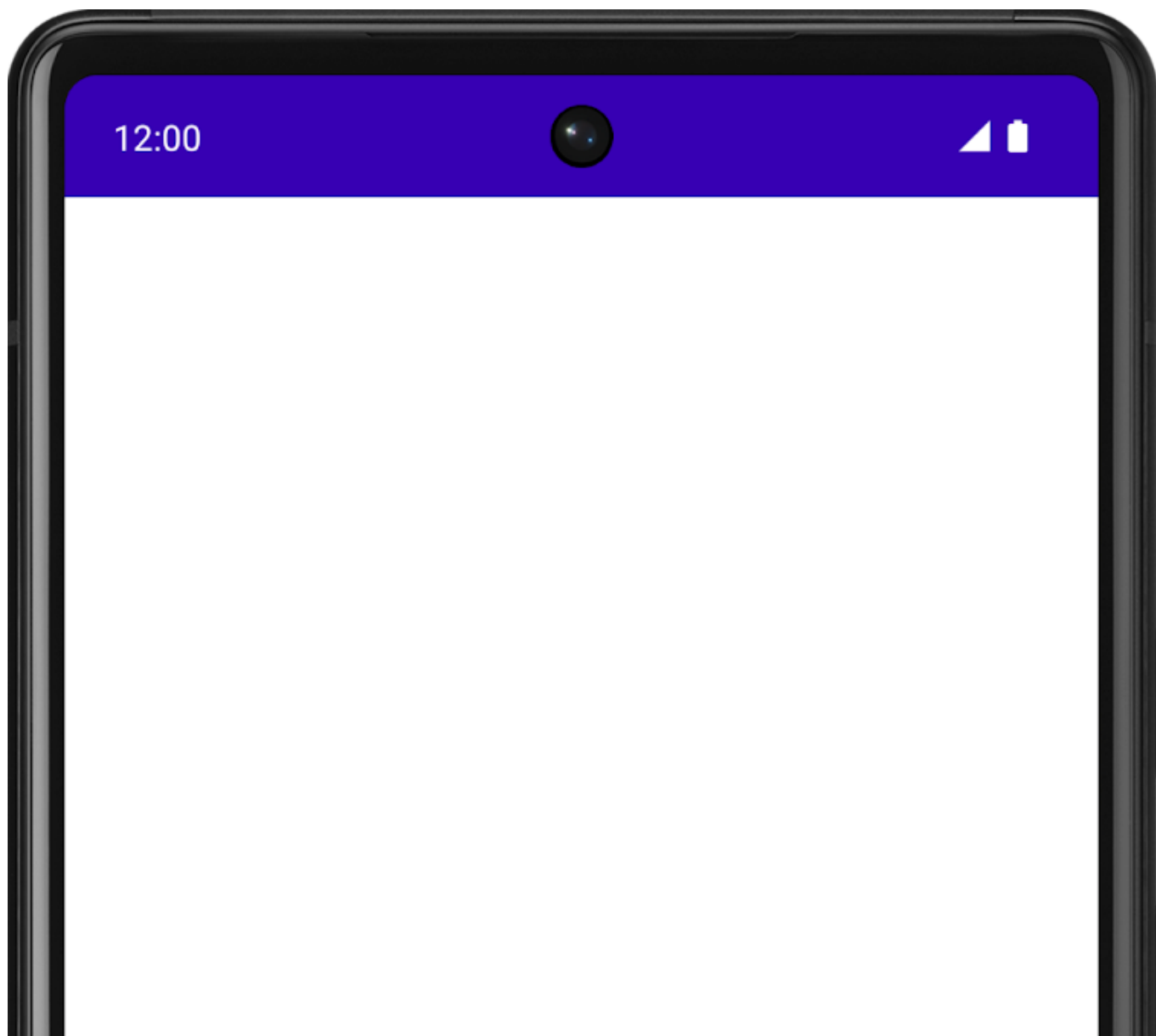
3. Загрузка стартового набора данных

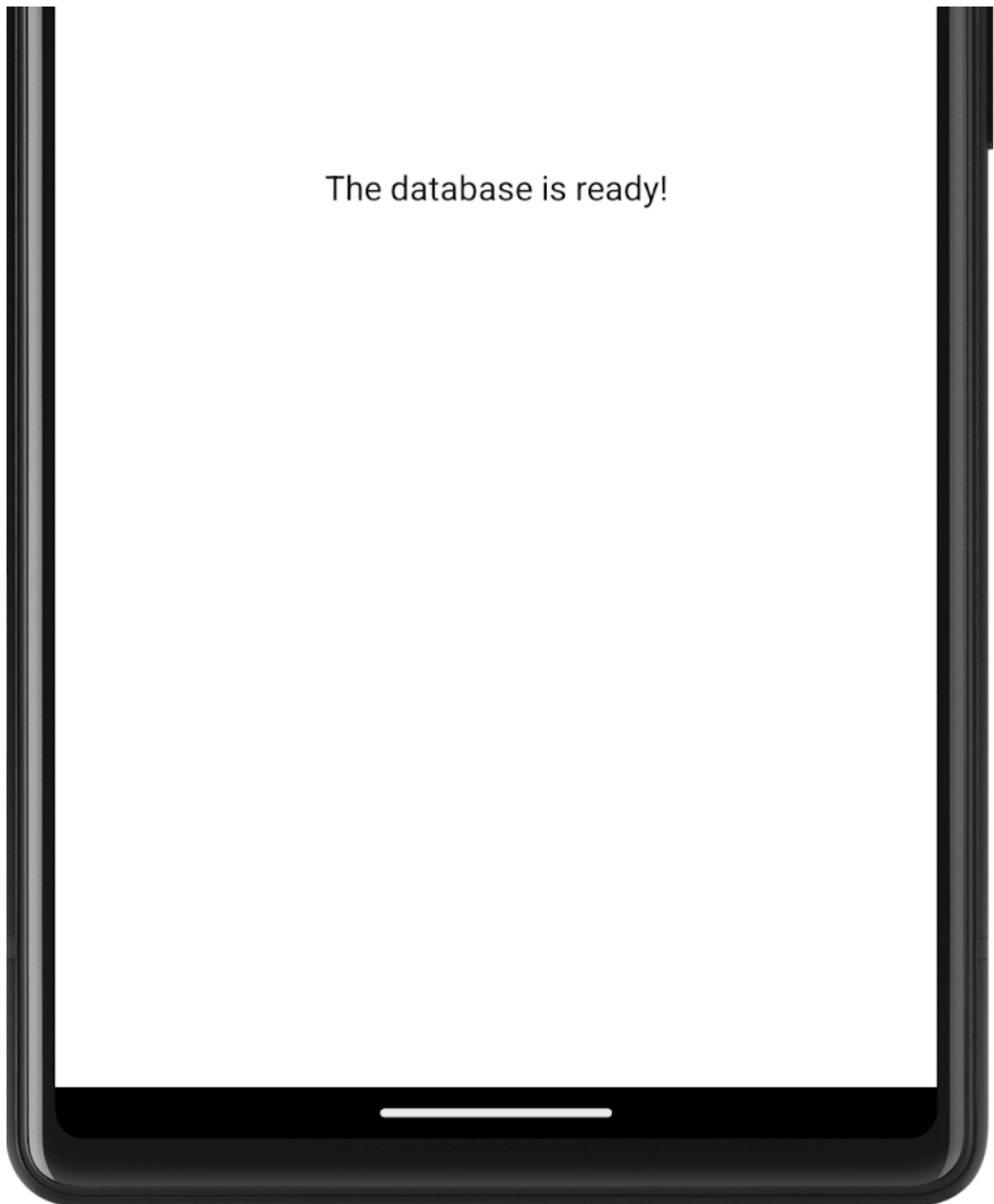
В качестве базы данных для этого коделаба используется гипотетическое приложение для работы с электронной почтой. В этом коделабе используются знакомые примеры, такие как сортировка и фильтрация почты, поиск по тексту темы или отправителя, чтобы продемонстрировать все мощные возможности, которые можно сделать с помощью SQL. Этот пример также поможет вам получить опыт работы с теми типами сценариев, которые вы можете встретить в приложении, прежде чем вы начнете работать с Room на следующем пути.

Загрузите начальный проект из ветки `compose` репозитория SQL Basics GitHub [здесь](#).

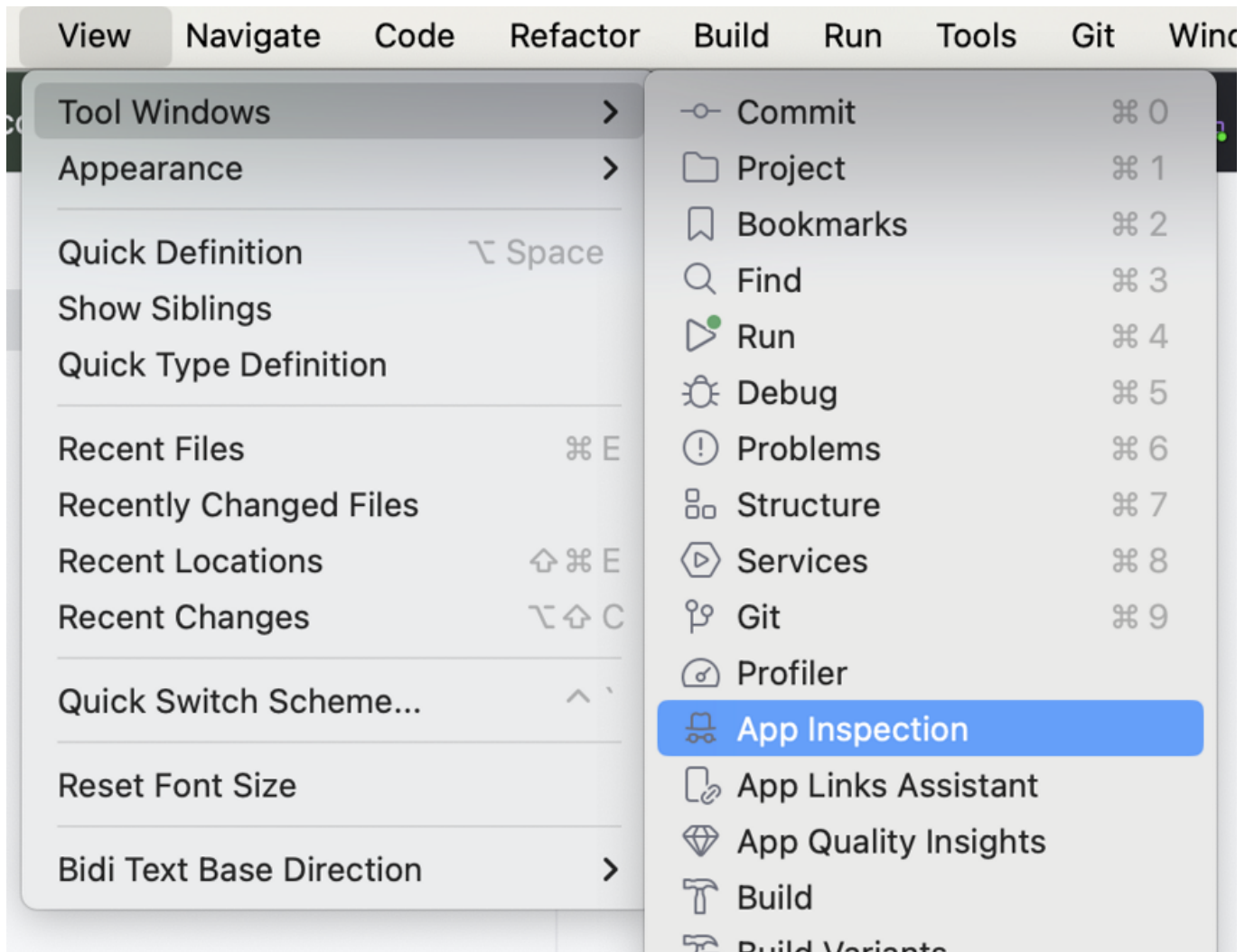
Использование инспектора баз данных Чтобы воспользоваться инспектором баз данных, выполните следующие действия:

Запустите приложение SQL Basics в Android Studio. При запуске приложения вы увидите следующее окно.

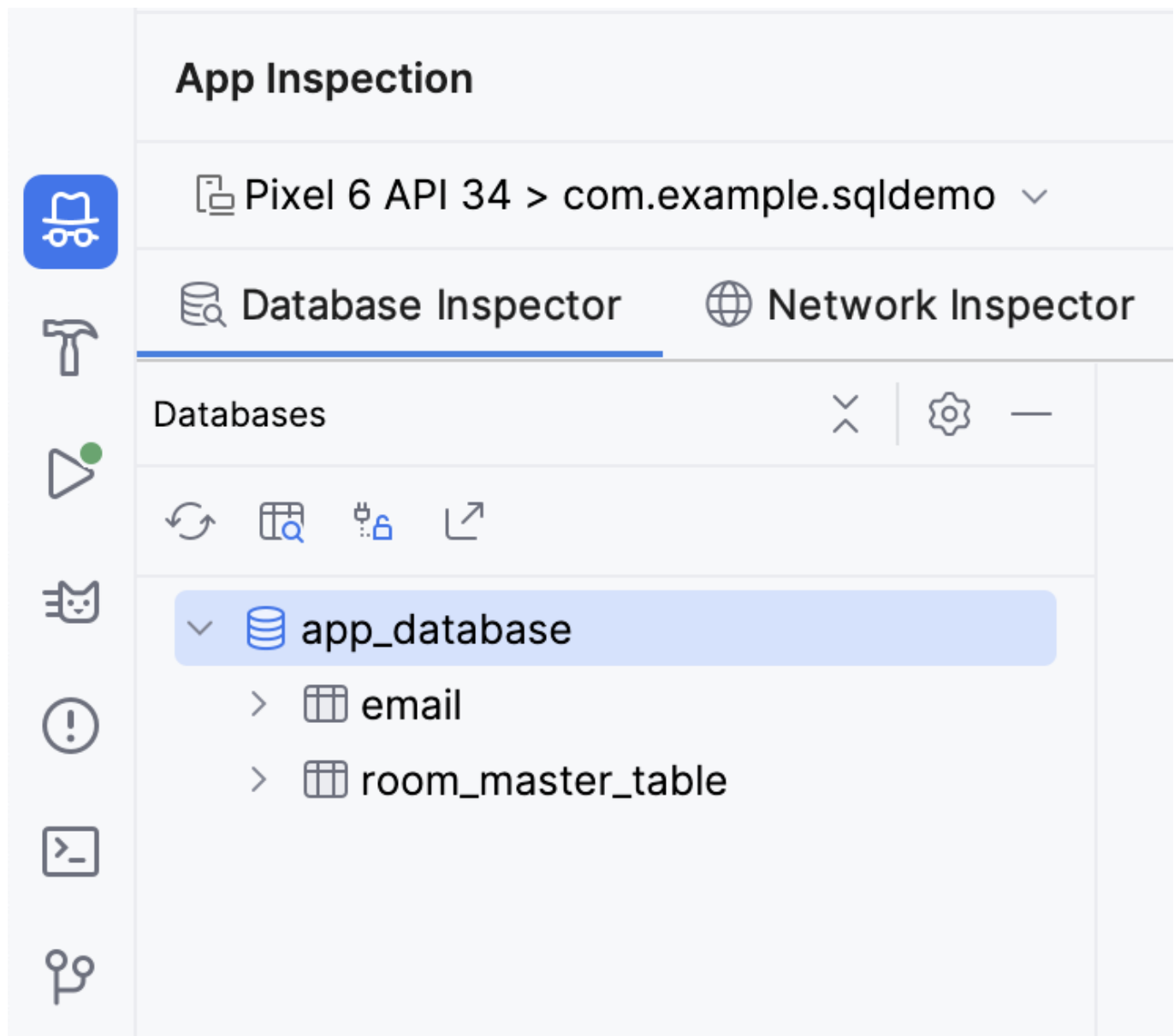




В Android Studio нажмите View > Tool Windows > App Inspection.



Теперь вы видите новую вкладку App Inspection с выбранной вкладкой Database Inspector. Есть еще две дополнительные вкладки, но они вам не понадобятся. Загрузка может занять несколько секунд, но затем вы увидите список слева с таблицами данных, которые можно выбрать для выполнения запросов.



Нажмите кнопку Open New Query Tab, чтобы открыть панель для запуска запроса к базе данных.

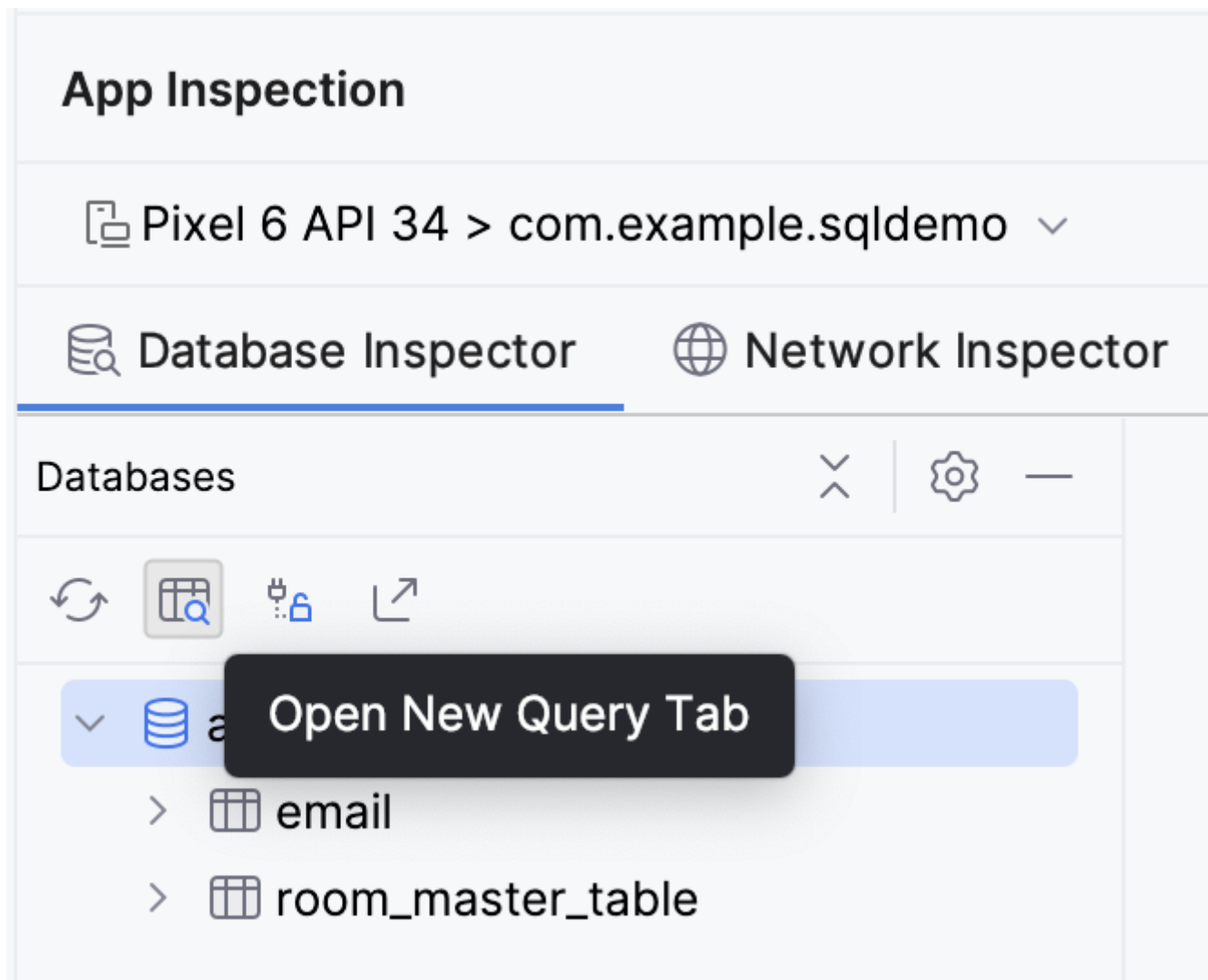
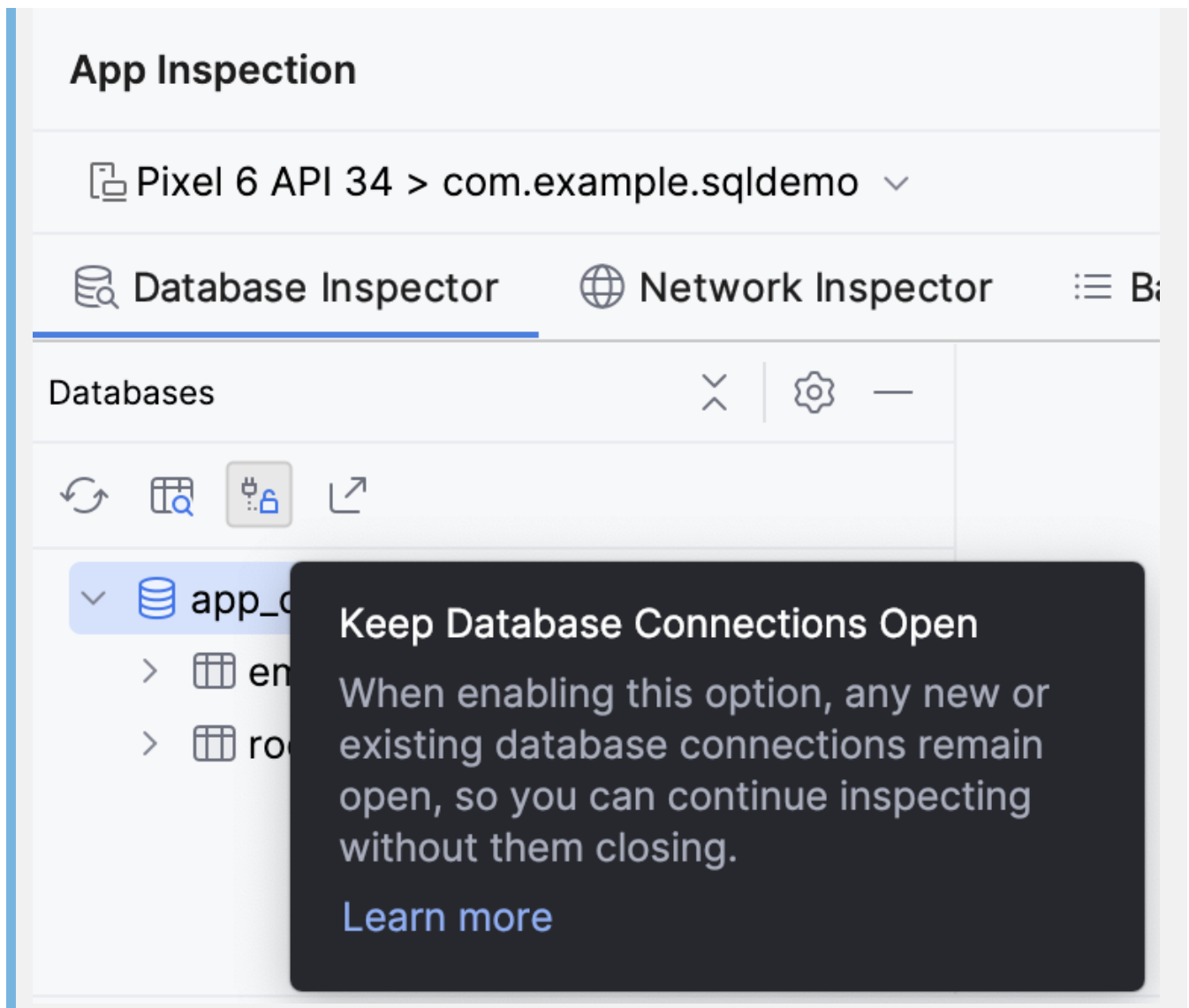


Таблица email содержит 7 столбцов:

id: Первичный ключ. тема: Тема письма. отправитель: Адрес электронной почты, с которого было отправлено письмо. папка: Папка, в которой можно найти сообщение, например «Входящие» или «Спам». звездочка: Отметил ли пользователь письмо звездочкой. прочитано: Прочитал ли пользователь письмо. получено: Временная метка, когда письмо было получено.

Совет: Нажмите кнопку Keep Database Connections Open c344609191760f79.png, чтобы продолжить взаимодействие с базой данных после выключения эмулятора.



4. Чтение данных с помощью оператора SELECT

Оператор SQL SELECT

Оператор SQL - иногда его называют запросом - используется для чтения или манипулирования базой данных.

Вы читаете данные из базы данных SQLite с помощью оператора SELECT. Простой оператор SELECT состоит из ключевого слова SELECT, затем имени столбца, затем ключевого слова FROM, затем имени таблицы. Каждый оператор SQL заканчивается точкой с запятой (👉).

SELECT **Column name** FROM **Table name** ;

Оператор SELECT может также возвращать данные из нескольких столбцов. Имена столбцов следует разделять запятой.

```
SELECT Column 1 , Column 2 FROM Table name ;
```

Если необходимо выбрать каждый столбец из таблицы, вместо имен столбцов используется символ подстановки (*).

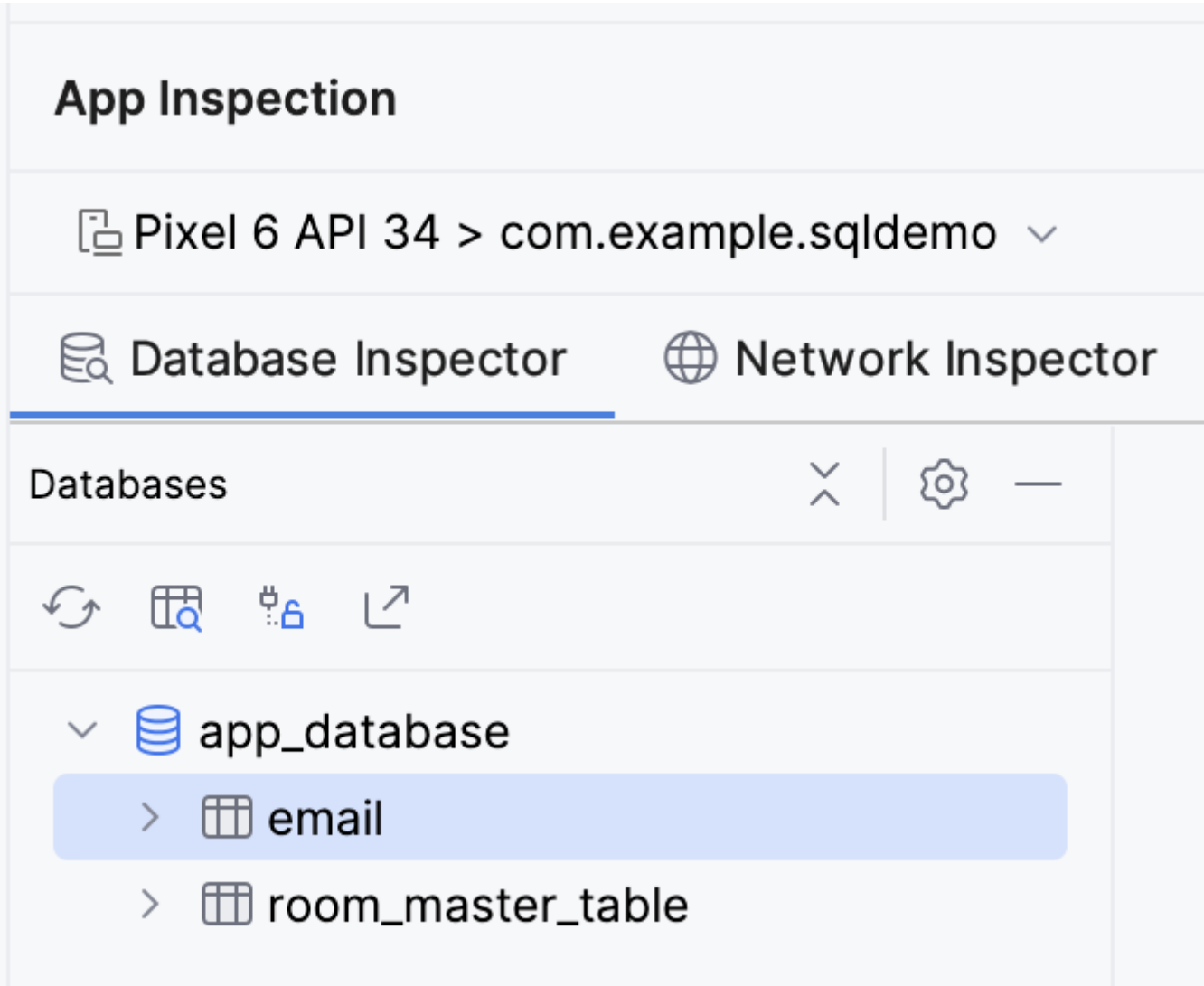
```
SELECT * FROM Table name ;
```

В любом случае простой оператор SELECT, подобный этому, возвращает каждую строку в таблице. Вам просто нужно указать имена столбцов, которые вы хотите вернуть.

Примечание: Несмотря на то, что по правилам каждый SQL-запрос должен заканчиваться точкой с запятой (🙄), некоторые редакторы, например, инспектор баз данных в Android Studio, позволяют опустить точку с запятой. На диаграммах в этом учебном пособии точка с запятой ставится в конце каждого полного SQL-запроса.

Чтение данных электронной почты с помощью оператора SELECT Одна из основных задач приложения для работы с электронной почтой - отображение списка сообщений. В базе данных SQL эту информацию можно получить с помощью оператора SELECT.

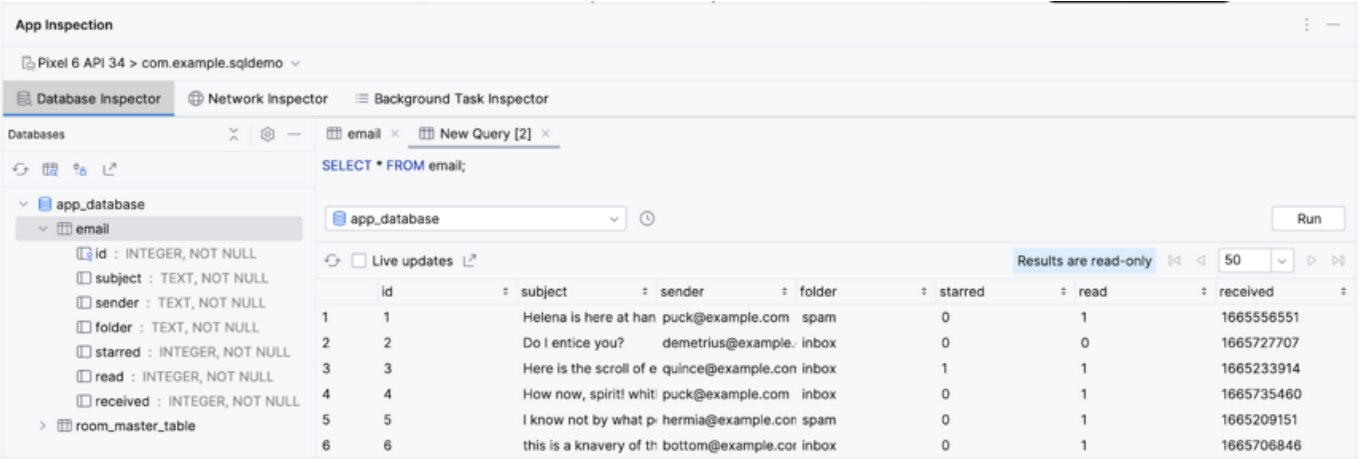
Убедитесь, что таблица email выбрана в инспекторе базы данных.



Сначала попробуйте выбрать каждый столбец из каждой строки таблицы email.

```
SELECT * FROM email;
```

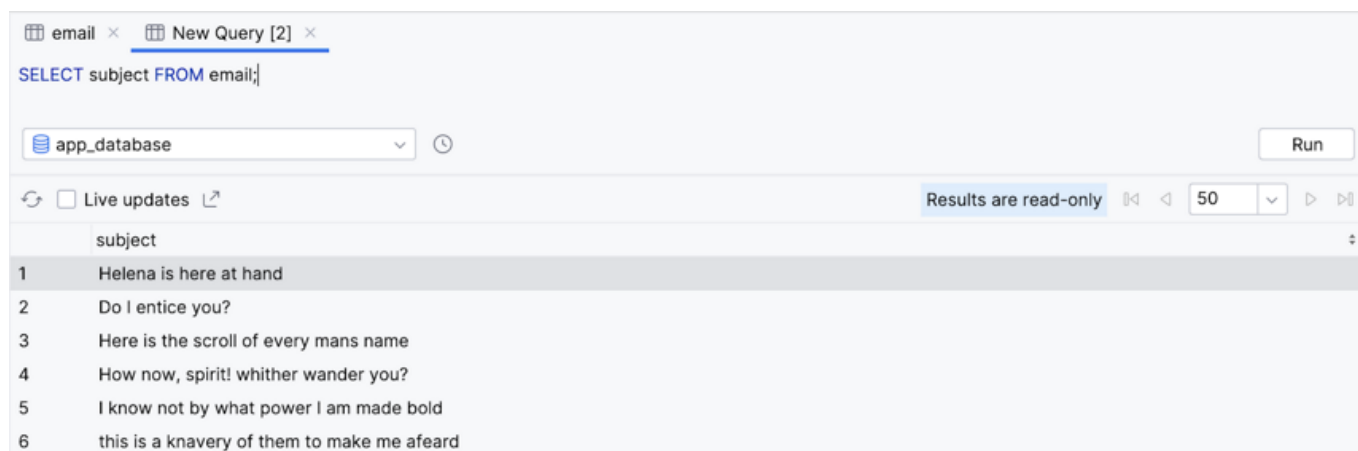
Нажмите кнопку Run в правом нижнем углу текстового поля. Обратите внимание, что возвращается вся таблица электронной почты.



Теперь попробуйте выбрать только тему для каждой строки.

```
SELECT subject FROM email;
```

Обратите внимание, что запрос снова возвращает все строки, но только для одного столбца.



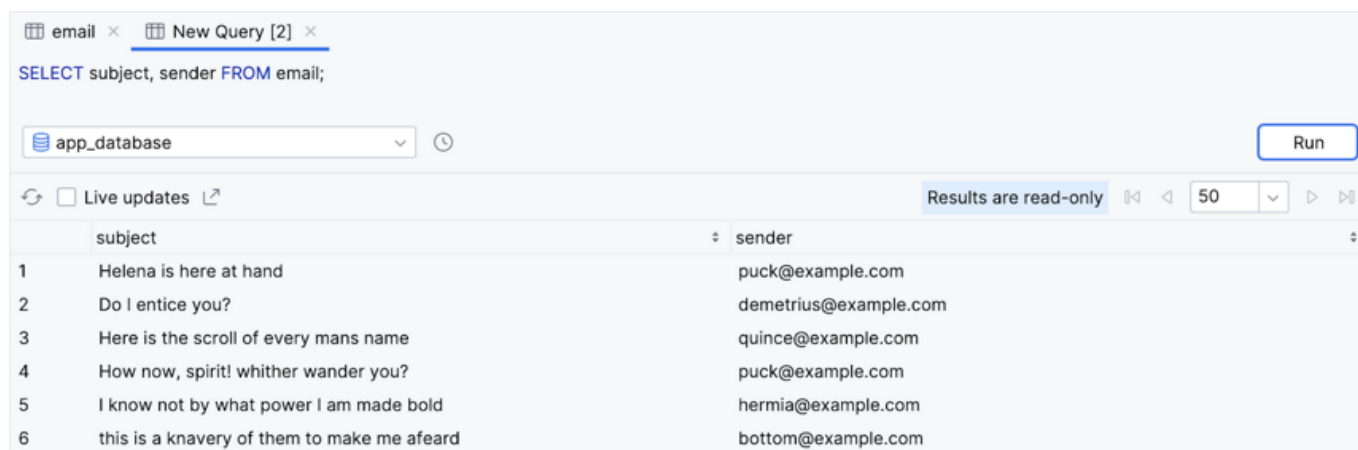
The screenshot shows a SQL query editor with a tab labeled 'email' and 'New Query [2]'. The query entered is 'SELECT subject FROM email;'. The database selected is 'app_database'. The results are displayed in a table with one column, 'subject', and six rows of data. The interface includes a 'Run' button, a 'Live updates' checkbox, and a 'Results are read-only' status bar.

	subject
1	Helena is here at hand
2	Do I entice you?
3	Here is the scroll of every mans name
4	How now, spirit! whither wander you?
5	I know not by what power I am made bold
6	this is a knavery of them to make me afeard

Вы также можете выбрать несколько столбцов. Попробуйте выбрать тему и отправителя.

```
SELECT subject, sender FROM email;
```

Обратите внимание, что запрос возвращает все строки таблицы email, но только значения столбцов subject и sender.



The screenshot shows the same SQL query editor with the query 'SELECT subject, sender FROM email;'. The results are displayed in a table with two columns, 'subject' and 'sender', and six rows of data. The interface is identical to the previous screenshot.

	subject	sender
1	Helena is here at hand	puck@example.com
2	Do I entice you?	demetrius@example.com
3	Here is the scroll of every mans name	quince@example.com
4	How now, spirit! whither wander you?	puck@example.com
5	I know not by what power I am made bold	hermia@example.com
6	this is a knavery of them to make me afeard	bottom@example.com

Поздравляем! Вы только что выполнили свой первый запрос. Неплохо, но считайте, что это только начало; «привет мир» SQL, если хотите.

Операторы SELECT могут быть гораздо более конкретными, добавляя условия для определения подмножества данных и даже изменяя форматирование вывода. В следующих разделах вы узнаете о часто используемых пунктах операторов SELECT и о том, как форматировать данные.

5. Использование операторов SELECT с агрегатными функциями и отличительными значениями

Сокращение столбцов с помощью агрегатных функций Операторы SQL не ограничиваются возвратом строк. SQL предлагает множество функций, которые могут выполнять операции или вычисления над определенным столбцом, например, находить максимальное значение или подсчитывать количество уникальных значений для определенного столбца. Эти функции называются агрегатными. Вместо того чтобы возвращать все данные определенного столбца, вы можете вернуть одно значение из определенного столбца.

Примеры агрегатных функций SQL включают следующее:

COUNT(): Возвращает общее количество строк, соответствующих запросу. SUM(): Возвращает сумму значений для всех строк в выбранном столбце. AVG(): Возвращает среднее значение-среднее значение всех значений в выбранном столбце. MIN(): Возвращает наименьшее значение в выбранном столбце. MAX(): Возвращает наибольшее значение в выбранном столбце. Вместо имени столбца можно вызвать агрегатную функцию и передать имя столбца в качестве аргумента между скобками.

SELECT **Aggregate function** (**Column name**)

Вместо того чтобы возвращать значение этого столбца для каждой строки таблицы, при вызове агрегатной функции возвращается одно значение.

Агрегатные функции могут быть эффективным способом выполнения вычислений над значением, когда вам не нужно читать все данные в базе данных. Например, вы можете захотеть найти среднее значение в столбце, не загружая всю базу данных в List и не делая это вручную.

Давайте посмотрим на некоторые агрегатные функции в действии на примере таблицы электронной почты:

Приложение может захотеть получить общее количество полученных писем. Это можно сделать с помощью функции COUNT() и символа подстановки (*).

```
``kt SELECT COUNT(*) FROM email;
```

Запрос возвращает одно значение. Вы можете сделать это полностью с помощью SQL-запроса, без какого-либо кода Kotlin для подсчета строк вручную.

Чтобы получить время самого последнего сообщения, можно использовать функцию MAX() для столбца received, поскольку самая последняя временная метка Unix - это наибольшее число.

```
SELECT MAX(received) FROM email;
```

Запрос возвращает один результат - самую последнюю временную метку в столбце received.

Фильтр дублирующихся результатов с помощью DISTINCT

Когда вы выбираете столбец, перед ним можно поставить ключевое слово DISTINCT. Такой подход может быть полезен, если вы хотите удалить дубликаты из результатов запроса.

Например, во многих почтовых приложениях есть функция автозаполнения адресов. Возможно, вы захотите включить все адреса, с которых вы получаете электронные письма, и отобразить их в виде списка.

Выполните следующий запрос, чтобы вернуть столбец «Отправитель» для каждой строки.

```
``SELECT sender FROM email;``
```

Обратите внимание, что результат содержит множество дубликатов. Это определенно не идеальный опыт!

Добавьте ключевое слово DISTINCT перед столбцом sender и выполните запрос заново.

```
``SELECT DISTINCT sender FROM email;``
```

Обратите внимание, что результат теперь намного меньше, а каждое значение уникально.

Перед именем столбца в агрегатной функции также можно поставить ключевое слово DISTINCT.

Допустим, вы хотите узнать количество уникальных отправителей в базе данных. Вы можете подсчитать количество уникальных отправителей с помощью агрегатной функции COUNT() и ключевого слова DISTINCT для столбца sender.

Выполните оператор SELECT, передав функции COUNT() значение DISTINCT sender.

```
``SELECT COUNT(DISTINCT sender) FROM email;``
```

Обратите внимание, что запрос сообщает нам, что существует 14 уникальных отправителей.

6. Фильтр запросов с помощью предложения WHERE

Многие почтовые приложения предлагают функцию фильтрации отображаемых сообщений по определенным критериям, таким как данные, поисковый запрос, папка, отправитель и т. д. Для таких случаев вы можете добавить в запрос SELECT предложение WHERE.

После имени таблицы в новой строке можно добавить ключевое слово WHERE, за которым следует выражение. При написании более сложных SQL-запросов принято выносить каждое предложение на новую строку для удобства чтения.

Этот запрос выполняет булеву проверку для каждой выбранной строки; если проверка возвращает true, то строка включается в результат запроса. Строки, для которых запрос возвращает false, не включаются в результат.

Например, в почтовом приложении могут быть фильтры для спама, корзины, черновиков или созданные пользователем фильтры. В следующих инструкциях это делается с помощью предложения WHERE:

Выполните оператор SELECT, чтобы вернуть все столбцы (*) из таблицы электронной почты, включая предложение WHERE для проверки условия `folder = 'inbox'`. Нет, это не опечатка: для проверки равенства в SQL используется одинарный знак равенства, а для представления строкового значения - одинарные, а не двойные кавычки.

```
``kt
SELECT * FROM email
WHERE folder = 'inbox';
```

Результат возвращает строки только для сообщений в папке «Входящие» пользователя.

New Query [2] ×

SELECT * FROM email
WHERE folder = 'inbox';

app_database

Run

☐ Live updates

Results are read-only

50

	id	subject	sender	folder	starred	read	received
1	2	Do I entice you?	demetrius@example.	inbox	0	0	1665727707
2	3	Here is the scroll of e	quince@example.com	inbox	1	1	1665233914
3	4	How now, spirit! whi	puck@example.com	inbox	0	1	1665735460
4	6	this is a knavery of th	bottom@example.cor	inbox	0	1	1665706846
5	7	let it be written in eig	bottom@example.cor	inbox	0	1	1665314896
6	8	Here, Peter Quince.	snout@example.com	inbox	0	1	1665507833

Примечание: Обратите особое внимание на операторы сравнения в SQL!

В отличие от Kotlin, оператор сравнения в SQL - это одинарный знак равенства (=), а не двойной знак равенства (==). Оператор неравенства (!=) такой же, как и в Kotlin. SQL также предоставляет операторы

сравнения $<$, $<=$, $>$ и $>=$.

Логические операторы в предложениях WHERE Клаузы SQL WHERE не ограничиваются одним выражением. Вы можете использовать ключевое слово AND, эквивалентное оператору Kotlin and (&&), чтобы включить только те результаты, которые удовлетворяют обоим условиям.

WHERE

First condition

AND

Second condition

В качестве альтернативы можно использовать ключевое слово OR, эквивалентное оператору Kotlin or (||), чтобы включить в результаты строки, удовлетворяющие любому из условий.

WHERE

First condition

OR

Second condition

Для удобства чтения можно также отрицать выражение с помощью ключевого слова NOT.

WHERE NOT

Condition

Многие приложения для работы с электронной почтой позволяют использовать несколько фильтров, например, показывать только непрочитанные письма.

Попробуйте использовать следующие более сложные условия WHERE для таблицы email:

Помимо возврата только сообщений в папке входящих сообщений пользователя, попробуйте также ограничить результаты непрочитанными сообщениями, если значение столбца read равно false.

```
SELECT * FROM email
WHERE folder = 'inbox' AND read = false;
```

Обратите внимание, что после выполнения запроса результаты содержат только непрочитанные письма в папке «Входящие» пользователя.

New Query [2] x

```
SELECT * FROM email
WHERE folder = 'inbox' AND read = false;
```

app_database

Run

Live updates

Results are read-only

	id	subject	sender	folder	starred	read	received
1	2	Do I entice you?	demetrius@example.	inbox	0	0	1665727707
2	18	The best in this kind ;	theseus@example.co	inbox	0	0	1665290247
3	22	Well roared, Lion.	demetrius@example.	inbox	0	0	1665209744
4	31	My legs can keep no	hermia@example.con	inbox	0	0	1665283102
5	43	A foolish heart, that I	helen@example.con	inbox	0	0	1665190298

Возвращает все письма, которые находятся в папке «Важные» ИЛИ отмечены звездочками (starred = true). Это означает, что результат включает письма в разных папках, если они отмечены звездочками.

```
SELECT * FROM email
WHERE folder = 'important' OR starred = true;
```

Наблюдайте за результатом.

New Query [2] x

```
SELECT * FROM email
WHERE folder = 'important' OR starred = true;
```

app_database

Run

Live updates

Results are read-only

	id	subject	sender	folder	starred	read	received
1	3	Here is the scroll of e	quince@example.con	inbox	1	1	1665233914
2	12	Never did mockers w	helen@example.con	spam	1	1	1665534357
3	16	Seeking sweet favou	oberon@example.cor	inbox	1	1	1665553578
4	21	You thief of love! wh	hermia@example.con	inbox	1	1	1665502228

Примечание: Вы также можете записать условие SQL NOT folder = 'spam' как folder != 'spam'.

Поиск текста с помощью LIKE Одна суперполезная вещь, которую можно сделать с помощью предложения WHERE, - это поиск текста в определенном столбце. Для этого нужно указать имя столбца, затем ключевое слово LIKE, а затем строку поиска.

WHERE **Column name** LIKE **Search string**

Строка поиска начинается с символа процента (%), затем следует текст для поиска (Search term), затем снова символ процента (%).

LIKE ' % **Search term** % '

Если вы ищете префикс - результаты, которые начинаются с указанного текста, - первый символ процента (%) исключается.

LIKE ' **Search term** % '

Кроме того, если вы ищете суффикс, опустите последний символ процента (%).

LIKE ' % **Search term** '

Существует множество вариантов использования текстового поиска в приложениях, например поиск писем, содержащих определенный текст в строке темы, или обновление предложений автозаполнения по мере ввода текста пользователем.

Следующие инструкции позволят вам использовать текстовый поиск при запросе к таблице электронной почты.

Персонажи Шекспира, как и персонажи нашей базы данных, любили говорить о дураках. Выполните следующий запрос, чтобы получить общее количество писем с текстом «дурак» в строке темы.

```
``kt SELECT COUNT(*) FROM email WHERE subject LIKE '%fool%';
```

Наблюдайте за результатом.

Выполните следующий запрос, чтобы вернуть все столбцы из всех строк, в которых тема заканчивается словом fool.

```
SELECT * FROM email WHERE subject LIKE '%fool';
```

Обратите внимание, что возвращаются две строки.

Выполните следующий запрос, чтобы вернуть отдельные значения столбца `sender`, начинающиеся с буквы `h`.

```
SELECT DISTINCT sender FROM email WHERE sender LIKE 'h%';
```

Обратите внимание, что запрос возвращает три значения: `helen@example.com` , `hyppolytus@example.com` и `hermia@example.com`.

7. Группировка, упорядочивание и ограничение результатов

Группировка результатов с помощью `GROUP BY`

Вы только что узнали, как использовать агрегатные функции и предложение `WHERE` для фильтрации и сокращения результатов. SQL предлагает несколько других предложений, которые могут помочь вам отформатировать результаты запроса. Среди них группировка, упорядочивание и ограничение результатов.

Вы можете использовать предложение `GROUP BY` для группировки результатов таким образом, чтобы все строки, имеющие одинаковое значение для данного столбца, были сгруппированы рядом друг с другом в результатах. Это предложение не изменяет результаты, а только порядок их возврата.

Чтобы добавить предложение `GROUP BY` в оператор `SELECT`, добавьте ключевое слово `GROUP BY`, за которым следует имя столбца, по которому вы хотите сгруппировать результаты.

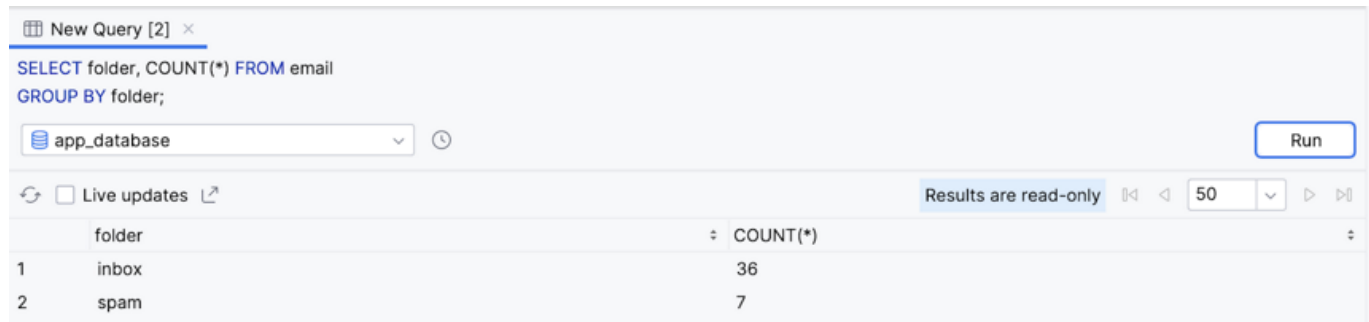
Чаще всего используется сочетание предложения `GROUP BY` с агрегатной функцией, чтобы разделить результат агрегатной функции по разным группам, например, по значениям столбцов. Вот пример. Представьте, что вы хотите получить количество писем в каждой папке: «Входящие», «Спам» и т. д. Вы можете выбрать как столбец папок, так и агрегатную функцию `COUNT()`, а также указать столбец папок в предложении `GROUP BY`.

Выполните следующий запрос, чтобы выбрать столбец папок и результат агрегатной

функции COUNT(). Используйте предложение GROUP BY, чтобы разделить результаты по значению в столбце папок.

```
``kt
SELECT folder, COUNT(*) FROM email
GROUP BY folder;
```

Наблюдайте за результатами. Запрос возвращает общее количество писем для каждой папки.



New Query [2] x

SELECT folder, COUNT(*) FROM email
GROUP BY folder;

app_database Run

Live updates Results are read-only 50

	folder	COUNT(*)
1	inbox	36
2	spam	7

Примечание: В предложении GROUP BY можно указать несколько столбцов, разделенных запятой, если вы хотите разделить каждую группу на дополнительные подгруппы, основанные на разных столбцах.

Сортировка результатов с помощью ORDER BY Вы также можете изменить порядок результатов запроса, отсортировав их с помощью предложения ORDER BY. Добавьте ключевое слово ORDER BY, затем имя столбца и направление сортировки.

ORDER BY

Column name

Sort direction

По умолчанию направлением сортировки является порядок возрастания, который можно опустить в предложении ORDER BY. Если вы хотите, чтобы результаты были отсортированы в порядке убывания, добавьте DESC после имени столбца.

Скорее всего, вы ожидаете, что приложение электронной почты будет показывать в первую очередь самые последние письма. Следующие инструкции позволят вам сделать это с помощью предложения ORDER BY.

Добавьте предложение ORDER BY для сортировки непрочитанных писем на основе столбца «Полученные». Поскольку по умолчанию используется порядок по возрастанию - младшие или старшие по возрасту, необходимо использовать ключевое слово DESC.

```
SELECT * FROM email
ORDER BY received DESC;
```

Понаблюдайте за результатом.

New Query [2] x

SELECT * FROM email
ORDER BY received DESC;

app_database

Run

☐ Live updates

Results are read-only

50

	id	subject	sender	folder	starred	read	received
1	4	How now, spirit! whi	puck@example.com	inbox	0	1	1665735460
2	2	Do I entice you?	demetrius@example.	inbox	0	0	1665727707
3	39	The very best at a be	demetrius@example.	inbox	0	1	1665724886
4	34	do I speak you fair?	demetrius@example.	inbox	0	1	1665720347
5	29	What, jealous Oberon	titania@example.com	inbox	0	1	1665711203
6	6	this is a knavery of th	bottom@example.cor	inbox	0	1	1665706846

Можно использовать предложение ORDER BY вместе с предложением WHERE. Скажем, пользователь хочет найти старые письма, содержащие текст fool. Можно отсортировать результаты, чтобы сначала отображались самые старые письма в порядке возрастания.

Выберите все письма, в теме которых содержится текст «дурак», и отсортируйте результаты по возрастанию. Поскольку порядок сортировки является возрастающим и используется по умолчанию, если не указан ни один порядок, использование ключевого слова ASC в предложении ORDER BY необязательно.

```
SELECT * FROM email
WHERE subject LIKE '%fool%'
ORDER BY received ASC;
```

Обратите внимание, что отфильтрованные результаты возвращаются с самым старым наименьшим значением в полученном столбце, показанным первым.

New Query [2] x

SELECT * FROM email
WHERE subject LIKE '%fool%'
ORDER BY received ASC;

app_database

Run

☐ Live updates

Results are read-only

50

	id	subject	sender	folder	starred	read	received
1	43	A foolish heart, that I	helena@example.com	inbox	0	0	1665190298
2	24	man is but a patched	bottom@example.cor	inbox	0	1	1665236168
3	25	Lord, what fools thes	puck@example.com	inbox	0	1	1665371560
4	16	Seeking sweet favou	oberon@example.cor	inbox	1	1	1665553578

Примечание: Если в одном запросе используются оба варианта, то предложение GROUP BY идет перед предложением ORDER BY.

Ограничение количества результатов с помощью LIMIT

До сих пор все примеры возвращали все результаты из базы данных, соответствующие запросу. Во многих случаях вам нужно отобразить только ограниченное количество строк из базы данных. Вы можете добавить в запрос предложение LIMIT, чтобы вернуть только определенное количество

результатов. Добавьте ключевое слово **LIMIT**, за которым следует максимальное количество строк, которые вы хотите вернуть. Если применимо, предложение **LIMIT** следует после предложения **ORDER BY**.

LIMIT

Max rows to return

В качестве опции можно включить ключевое слово **OFFSET**, за которым следует другое число, обозначающее количество строк, которые нужно «пропустить». Например, если вам нужны следующие десять результатов после первых десяти, но вы не хотите возвращать все двадцать результатов, вы можете использовать **LIMIT 10 OFFSET 10**.

LIMIT

Max rows to return

OFFSET

Rows to skip

В качестве опции вы можете включить ключевое слово **OFFSET**, за которым следует другое число, обозначающее количество строк, которые нужно «пропустить». Например, если вам нужны следующие десять результатов после первых десяти, но вы не хотите возвращать все двадцать результатов, вы можете использовать **LIMIT 10 OFFSET 10**.

LIMIT

Max rows to return

OFFSET

Rows to skip

В приложении вы можете захотеть быстрее загружать электронные письма, возвращая только первые десять писем в папке входящих сообщений пользователя. Затем пользователи могут прокручивать страницу, чтобы просмотреть последующие письма. В следующих инструкциях для достижения такого поведения используется предложение **LIMIT**.

Выполните следующий оператор **SELECT**, чтобы получить все письма в папке «Входящие» пользователя в порядке убывания и ограничиться первыми десятью результатами.

```
``kt SELECT * FROM email WHERE folder = 'inbox' ORDER BY received DESC LIMIT 10;
```

Обратите внимание, что возвращается только десять результатов.

Измените и повторно выполните запрос, включив в него ключевое слово **OFFSET** со значением 10.

```
SELECT * FROM email WHERE folder = 'inbox' ORDER BY received DESC LIMIT 10 OFFSET 10;
```

Запрос возвращает десять результатов в порядке убывания. Однако запрос пропускает первый набор из десяти результатов.

8. Вставка, обновление и удаление данных в базе данных

Вставка данных в базу данных

Помимо чтения из базы данных, существуют различные операторы SQL для записи в базу данных. Ведь данные должны как-то туда попасть, не так ли?

Добавить новую строку в базу данных можно с помощью оператора INSERT. Оператор INSERT начинается с INSERT INTO, за которым следует имя таблицы, в которую вы хотите вставить новую строку. Ключевое слово VALUES появляется на новой строке, за которым следует набор круглых скобок, содержащих список значений, разделенных запятыми. Значения нужно перечислять в том же порядке, что и столбцы базы данных.

Представьте, что пользователь получил новое письмо, и нам нужно сохранить его в базе данных нашего приложения. Мы можем использовать оператор INSERT для добавления новой строки в таблицу email.

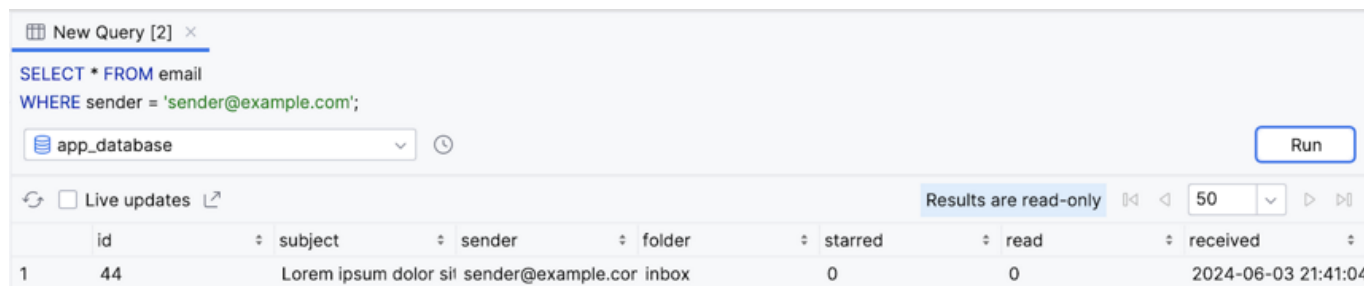
Выполните оператор INSERT со следующими данными для нового письма. Поскольку письмо новое, оно непрочитанное и первоначально появляется в папке «Входящие». Для столбца id указано значение NULL, что означает, что id будет автоматически сгенерирован из следующего доступного целого числа с автоинкрементацией.

```
INSERT INTO email VALUES ( NULL, 'Lorem ipsum dolor sit amet', 'sender@example.com', 'inbox', false, false, CURRENT_TIMESTAMP );
```

> Примечание: CURRENT_TIMESTAMP - это специальная переменная, которая при выполнении запроса заменяется на текущее время в UTC, что удобно при вставке новых строк!

Обратите внимание, что результат вставляется в базу данных с идентификатором 44.

```
`q1
SELECT * FROM email
WHERE sender = 'sender@example.com';
```



The screenshot shows a SQL query editor with the following content:

```
SELECT * FROM email
WHERE sender = 'sender@example.com';
```

The database selected is `app_database`. The results are read-only. The query returns one row of data:

	id	subject	sender	folder	starred	read	received
1	44	Lorem ipsum dolor sit	sender@example.com	inbox	0	0	2024-06-03 21:41:04

Обновление существующих данных в базе данных После того как вы вставили данные в таблицу, вы можете изменить их позже. Для этого можно обновить значение одного или нескольких столбцов с помощью оператора UPDATE. Оператор UPDATE начинается с ключевого слова UPDATE, за которым следует имя таблицы, а затем предложение SET.

UPDATE

Table name

SET

Sets of columns and values ;

Предложение SET состоит из ключевого слова SET, за которым следует имя столбца, который необходимо обновить.

SET

First column

=

First value ,

Second column

=

Second value ,

...

Оператор UPDATE часто включает предложение WHERE для указания одной или нескольких строк, которые необходимо обновить с помощью указанной пары столбец-значение.

UPDATE

Table name

SET

Sets of columns and values

WHERE

Condition(s) ;

Если, например, пользователь хочет пометить электронное письмо как прочитанное, для обновления базы данных используется оператор UPDATE. Следующие инструкции позволяют пометить электронное письмо, вставленное в предыдущем шаге, как прочитанное.

Выполните следующий оператор UPDATE, чтобы установить для строки с идентификатором 44 значение столбца read равным true.

```
``ql UPDATE email SET read = true WHERE id = 44;
```

Запустите оператор SELECT для этой конкретной строки, чтобы проверить результат.

```
````ql  
SELECT read FROM email
WHERE id = 44;
```

Обратите внимание, что значение столбца read теперь равно 1 для «истинного» значения, а не 0 для «ложного».


New Query [2] ×

---

**SELECT** read **FROM** email

**WHERE** id = 44;


app\_database
⌵
🕒

↺
☐ Live updates
 ↗

	read
1	1

Удаление строки из базы данных Наконец, вы можете использовать оператор SQL DELETE для удаления одной или нескольких строк из таблицы. Оператор DELETE начинается с ключевого слова DELETE, за которым следует ключевое слово FROM, затем имя таблицы, после чего следует предложение WHERE, указывающее, какую строку или строки вы хотите удалить.

DELETE FROM **Table name**

WHERE **Condition** ;

В следующих инструкциях используется оператор DELETE для удаления из базы данных ранее вставленной и впоследствии обновленной строки.

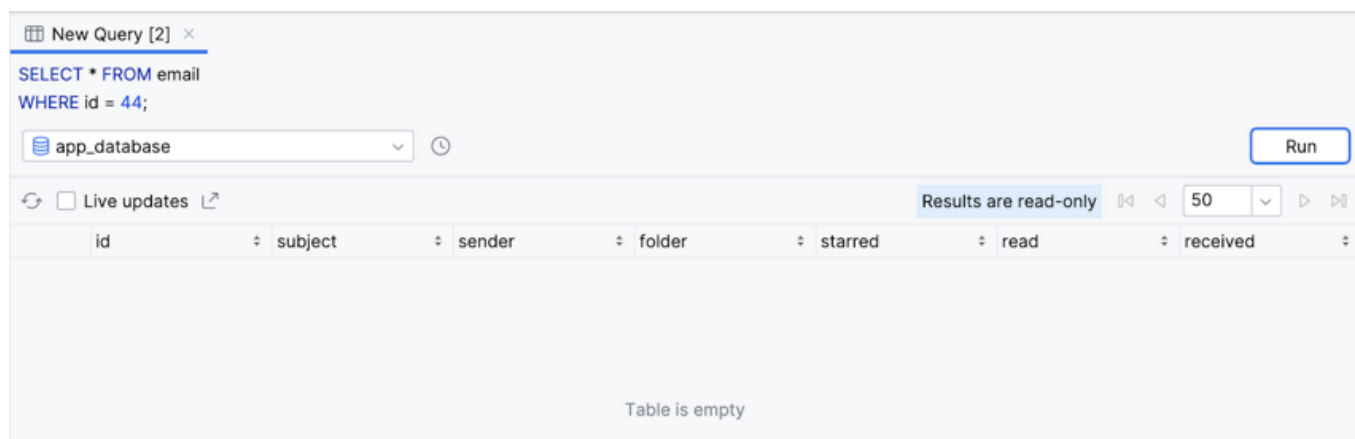
Выполните следующий оператор DELETE, чтобы удалить из базы данных строку с идентификатором 44.

```
``sql DELETE FROM email WHERE id = 44;
```

Проверьте изменения с помощью оператора SELECT.

```
``sql
SELECT * FROM email
WHERE id = 44;
```

Обратите внимание, что строка с идентификатором 44 больше не существует.



## Резюме

Поздравляем! Вы многому научились! Теперь вы можете считывать данные из базы данных с помощью операторов SELECT, включая предложения WHERE, GROUP BY, ORDER BY и LIMIT для фильтрации результатов. Вы также узнали о часто используемых агрегатных функциях, ключевом слове DISTINCT для указания уникальных результатов и ключевом слове LIKE для выполнения текстового поиска по значениям в столбце. Наконец, вы узнали, как выполнять INSERT, UPDATE и DELETE строк в таблице данных.

Эти навыки можно напрямую перенести в Room, и, зная SQL, вы будете более чем готовы к работе с данными в своих будущих приложениях.

Синтаксис оператора SELECT:

```

SELECT Columns or aggregate functions FROM Table name
WHERE Conditions
GROUP BY Columns
ORDER BY Column name Sort order
LIMIT Number of rows ;

```